

Project Proposal: Rebuilding the DyNetKAT-Zoo Benchmark Generator for the RaceLoom Journal Artifact

1 Current System Understanding

1.1 RaceLoom

RaceLoom consumes JSON network models and JSON forwarding-property files. Its command-line interface accepts an SDN model file, a forwarding-properties file, and options such as trace depth, strategy, thread count, and verbosity.

A RaceLoom model has the following high-level shape:

```
{
  "Switches": {
    "SW0": {
      "InitialFlowTable": "...",
      "DirectUpdates": [
        {"Channel": "...", "Policy": "..."}
      ],
      "RequestedUpdates": [
        {
          "RequestChannel": "...",
          "RequestPolicy": "...",
          "ResponseChannel": "...",
          "ResponsePolicy": "..."
        }
      ]
    }
  },
  "Links": "...",
  "RecursiveVariables": {
    "C0": "..."
  },
  "Controllers": ["C0"],
  "OtherChannels": []
}
```

Important compatibility facts are that RaceLoom expects a non-empty set of switches and controllers; Each controller listed in **Controllers** must be declared in **RecursiveVariables**; Switch update/request channels must be unique across switches; **DirectUpdates** currently contain only **Channel** and **Policy**; the RaceLoom schema does not include an **Append** field; and RaceLoom internally builds a symbolic “big switch” from the per-switch JSON representation. Therefore, the generator should emit per-switch JSON, not necessarily pre-collapse the network into a single **BigSW** object.

This last point is especially important. Although the paper discusses a one-big-switch abstraction,

RaceLoom itself already performs this abstraction internally. The input format can remain per-switch, preserving channel-to-switch metadata that is later used during race reconstruction.

1.2 RaceLoom Analysis Pipeline

RaceLoom’s execution pipeline is approximately as follows: Parse the JSON model into a Pydantic model; Convert the JSON model into a Maude representation; Generate symbolic traces up to a depth bound using DFS, BFS, or parallel BFS; Track causal relationships using vector clocks; Identify racing transitions; Reconstruct network policies at racing points; Use KATch to check whether the race is harmful with respect to the relevant safety property; and Export raw and graphical harmful traces and final statistics.

The trace analyzer contains an important limitation: it explicitly does not account for policies or flow rules appended to a flow table. Therefore, the journal-ready generator should avoid append-style updates unless RaceLoom itself is extended to support them soundly.

1.3 DyNetKAT-Zoo Generator

The current generator is Go-based and contains the following core concepts: GraphML parsing from Topology Zoo files; A deterministic ordered weighted undirected graph implementation; Network construction with switches, links, hosts, ports, controllers, flow rules, and flow tables; Scenario behaviors such as pairwise host connection, outside-host connection, and link-cost changing; and Encoders for RaceLoom JSON and LaTeX-style output.

The current `main.go` is essentially a benchmark script. It hard-codes the input directory, output directory, random seed, controller count, scenario, and encoder. This is not appropriate for a reusable journal artifact.

The current generator also contains two JSON encoder styles. **Whole-network update encoder.** This encoder emits one BigSW-style switch and folds the network into whole-network policies. It is close to the paper’s abstraction, but it ignores flow-rule sequences; and **Per-switch update encoder.** This encoder emits RaceLoom-style per-switch JSON with explicit `Switches`, `Links`, `RecursiveVariables`, and `Controllers`. This is better aligned with RaceLoom’s actual input schema and existing benchmark files.

2 Project Goal

The main goal is to produce a journal-quality benchmark-generation framework for RaceLoom. The generator should be **Correct**: generated JSON must validate against RaceLoom’s model schema and match RaceLoom’s semantic assumptions; **Reproducible**: generation must be deterministic for fixed inputs and seeds; **Documented**: users should understand how topologies, hosts, controllers, flow tables, links, updates, and properties are generated; **Extensible**: new scenarios and controller-assignment strategies should be easy to add; **Integrated**: the generator should optionally run RaceLoom and collect results; and **Artifact-ready**: the repository should include tests, examples, a quickstart, and benchmark reproduction scripts.

3 Design Alternatives

3.1 Alternative A: Minimal Cleanup of the Existing Go Generator

This option keeps the current Go code and makes only minimal changes: add a README; clean comments and naming; expose a few command-line flags; add limited tests; and ensure output works with RaceLoom.

3.2 Alternative B: Full Refactor of the Existing Go Generator

This option keeps Go but significantly restructures the code: convert `main.go` into a proper CLI; separate GraphML loading, graph construction, network construction, scenarios, encoding, validation, and output; standardize on the per-switch RaceLoom JSON backend; rewrite or update stale tests; and document all supported scenarios.

3.3 Alternative C: Ground-Up Python Generator

This option builds a new Python-based generator while preserving the Go generator as a reference implementation during reconstruction.

Pros: Aligns with RaceLoom’s implementation language; Can share or mirror RaceLoom’s Pydantic JSON schema; Makes end-to-end integration easier: generate, validate, run RaceLoom, collect stats; Easier to build a unified artifact workflow; Easier for future contributors to understand the full RaceLoom ecosystem in one language; and Better suited to building a flexible benchmark framework rather than a one-off converter.

Cons: Highest initial implementation effort; Requires careful reconstruction of deterministic ordering, graph traversal, port assignment, shortest-path tie-breaking, and scenario semantics; Risk of subtle differences from existing benchmark outputs; and Requires disciplined validation against the existing Go generator and RaceLoom.

3.4 Recommendation

I recommend **Alternative C**: build a new Python-based generator from the ground up, while preserving the existing Go generator as an oracle/reference until the new generator reproduces the expected outputs.

This choice is justified because the project goal is not merely to clean up an old script, but to produce an all-encompassing, fully functional journal artifact component. The new generator should be a reproducible benchmark construction framework for RaceLoom, not just a Topology Zoo converter.

4 Proposed Architecture

The proposed Python package could have the following structure:

```
dynetkat_zoo/  
  cli.py  
  
  graphml/  
    loader.py  
    topology_zoo.py
```

```
model/  
  graph.py  
  network.py  
  switch.py  
  host.py  
  controller.py  
  flow.py  
  policy.py  
  
scenarios/  
  base.py  
  diameter_pairwise.py  
  outside_hosts.py  
  link_cost_change.py  
  manual.py  
  
assignment/  
  controllers.py  
  hosts.py  
  
encoders/  
  raceloom_json.py  
  latex.py  
  debug.py  
  
properties/  
  generator.py  
  
validation/  
  raceloom_schema.py  
  
runner/  
  raceloom.py  
  
tests/  
  fixtures/  
  golden/
```

4.1 Pipeline

The generator pipeline should be explicit:

Stage	Responsibility
GraphML loading	Read Topology Zoo GraphML files and normalize topology names.
Deterministic graph model	Preserve stable node/edge ordering for reproducibility.
Network construction	Assign switches, links, ports, hosts, and initial flow tables.
Controller assignment	Assign controllers to switches according to a named strategy.
Scenario generation	Generate updates for a specific benchmark scenario.
Encoding IR	Build a neutral intermediate representation of flow tables, updates, links, and channels.
RaceLoom JSON encoding	Emit JSON matching RaceLoom’s accepted schema.
Property generation	Emit matching forwarding-property JSON.
Validation	Validate generated models against RaceLoom’s schema.
Execution	Optionally invoke RaceLoom and collect final stats.

4.2 Canonical Backend

The canonical backend should be the RaceLoom per-switch JSON format. Although the paper uses a big-switch abstraction, RaceLoom already constructs the big switch internally. Therefore, the generator should preserve per-switch structure in JSON and allow RaceLoom to collapse it.

This preserves: switch-to-channel mapping; deterministic switch ordering; per-switch flow-table updates; compatibility with existing benchmark files; and RaceLoom’s metadata reconstruction logic.

5 Scenario Design

The generator should support multiple scenarios, but the initial implementation should clearly distinguish the primary journal benchmark path from secondary or exploratory scenarios. The first and most important scenario is the **diameter pairwise host-connection scenario**. In this scenario, the generator selects pairs of hosts attached to switches that are far apart in the topology, typically using the topology diameter or a farthest-apart switch pair, and then generates controller updates that install forwarding behavior along shortest paths between those hosts. This scenario is attractive because it closely matches the current active behavior in the existing generator, scales naturally with topology size and diameter, and produces meaningful path-based flow-table updates without requiring complicated assumptions about traffic classes or application logic. It is also relatively easy to explain in the journal artifact: given a real topology, choose distant endpoints, synthesize forwarding updates, and analyze the resulting controller-switch races. Its main drawback is that it may not exercise every part of RaceLoom, especially requested updates, packet-in/packet-out behavior, and more dynamic controller interactions. If every controller is allowed to control every switch, it may also be somewhat abstract compared with realistic controller partitioning. Despite these limitations, this should be the primary benchmark scenario because it is the closest to the existing Topology Zoo evaluation path, the easiest to make reproducible, and the best starting point for validating the rebuilt generator against current benchmark outputs.

A second useful scenario is the **outside-host connection scenario**. Here, the network begins with an initial set of connected hosts and already-installed forwarding rules, after which new outside

hosts are introduced and controllers generate updates to connect those new hosts to the existing network. This scenario is valuable because it models a natural kind of SDN evolution: the network is not empty, and the controller is reacting to the addition of new communication endpoints. It can therefore produce a clearer before-and-after story than the diameter pairwise scenario, since some forwarding behavior is already present and the generated updates extend that behavior. The downside is that it depends more heavily on configuration choices, such as the number of initial hosts, the number of outside hosts, and how those hosts are placed. In the current Go implementation, this scenario also assumes that there are enough initial hosts to connect before outside hosts are added, so it is not compatible with configurations where the initial host count is zero. Because of that added complexity, this scenario should be preserved and eventually implemented in the new generator, but it should not be the first scenario used to validate the rebuilt framework. It is best treated as a secondary benchmark once the main diameter pairwise pipeline is stable.

A third scenario is the **link-cost change scenario**. In this case, the generator first constructs an initially connected network, then changes link costs, recomputes shortest paths, and generates controller updates that reflect the new routing decisions. This scenario is appealing because it models a realistic source of network updates: routing changes caused by shifts in link weights, congestion, failures, or administrative policy. It can produce interesting policy churn and may expose races that are different from those produced by simply connecting new host pairs. However, it is also more difficult to make clean and reproducible. Random link-cost changes may fail to change any shortest paths, requiring retries, and the behavior may vary substantially depending on the redundancy and structure of each topology. This makes it harder to compare results across Topology Zoo networks. For that reason, link-cost changing should be viewed as an extended or experimental scenario. It is worth supporting in the long-term framework, especially if the journal version wants to demonstrate broader applicability, but it should come after the core generator, schema validation, and main benchmark scenario are complete.

A fourth scenario is the **manual small-example scenario**. Instead of deriving everything from a Topology Zoo graph, this scenario would allow small hand-written networks, controller behaviors, and safety properties to be expressed directly. This is not intended to be the main large-scale benchmark mode, but it is extremely useful for testing, debugging, and documentation. It would make it easier to reproduce examples such as the firewall case and the independent-controller case, and it would provide compact models that can be included in the README, used in unit tests, and run quickly as smoke tests. The main limitation is that manual examples do not demonstrate scalability by themselves and require a separate input format or configuration style. Nevertheless, this scenario should be implemented early because it supports the rest of the project: it gives us small, understandable cases for validating the RaceLoom JSON encoder, property generation, controller update semantics, and end-to-end RaceLoom execution.

6 Controller Assignment Strategies

Controller assignment should be separated from scenario behavior. The current generator mixes these concerns: some scenarios use random split, while pairwise host connection uses no split.

The redesigned generator should support **all-switches**: every controller controls every switch; **random-split**: switches are randomly partitioned among controllers; **round-robin**: switches are assigned deterministically across controllers; and **from-file**: user supplies a controller-to-switch assignment.

This separation improves reproducibility and makes scenario comparisons easier.

7 Property Generation

The generator should be able to produce forwarding-property files as well as model files. RaceLoom properties are keyed by race type and contain an expression; and a boolean indicating whether packets satisfying the expression are allowed.

The generator should support **generic-drop**: e.g., `@Network. (dst=999) with AllowsPackets=false;` **no-src-to-dst**: topology-specific host reachability properties; and **from-file**: user-provided property files.

Property generation should be explicit because the safety property is part of the benchmark definition. A model without a matching property file is not a complete RaceLoom experiment.

8 Append Updates: Scope Decision

The current generator contains support for flow-rule sequences and append-like direct updates. However, RaceLoom’s model schema does not include **Append**, and the analyzer explicitly states that it does not account for appended flow rules.

Therefore, the journal-ready generator should initially avoid append updates and emit full flow-table replacement updates. Append semantics should be treated as future work unless we also extend RaceLoom.

Adding append support would require extending the JSON schema; extending the Maude model construction; extending trace metadata; updating policy reconstruction; updating harmful-race checks; and adding new tests and examples.

This is potentially valuable, but it is a RaceLoom feature extension, not merely a generator feature.

9 Validation Strategy

Validation should happen at multiple levels.

9.1 Schema Validation

Every generated RaceLoom model should validate against RaceLoom’s JSON schema. If the generator is written in Python, it can directly import or mirror the Pydantic models.

9.2 Determinism Validation

For a fixed topology, seed, scenario, and controller assignment, repeated generation should produce identical JSON or at least semantically equivalent JSON with stable ordering.

9.3 Golden Tests

A small set of known outputs should be stored as golden files. Suggested fixtures include **Arpanet196912** with seed 15; one medium topology such as **Abilene**; a small manual topology; a firewall-style example; and an independent-controller-style example.

9.4 RaceLoom Smoke Tests

For small generated models, the test suite should run RaceLoom at low depth and verify that the model parses; Maude trace generation runs; KATch property checking runs; stats are produced; and harmful trace output directories are created when applicable.

10 Implementation Plan

10.1 Phase 0: Specification

Deliverable: SPEC.md.

The specification should define: RaceLoom JSON schema; property schema; supported NetKAT fragment; channel naming rules; switch ordering rules; controller variable naming rules; scenario semantics; determinism guarantees; and unsupported semantics, especially append updates.

10.2 Phase 1: Reconstruct Existing Model Semantics

Deliverables: notes mapping current Go concepts to new Python concepts; reference outputs from the current Go generator; and tests documenting expected behavior.

Tasks: reconstruct GraphML parsing behavior; reconstruct deterministic graph ordering; reconstruct port assignment; reconstruct host placement on farthest-apart switches; reconstruct controller update generation; reconstruct NetKAT string generation; and compare against existing RaceLoom benchmark JSON.

10.3 Phase 2: Build Python Generator Core

Deliverables: Python package skeleton; graph model; network model; flow-rule and flow-table model; NetKAT policy model; and RaceLoom JSON encoder.

10.4 Phase 3: Implement Main Scenario

Deliverable: `diameter-pairwise` scenario.

Tasks: place hosts on farthest-apart switches; compute paths; generate flow-table replacement updates; support alternating direction; support configurable controller count; and support configurable controller assignment.

10.5 Phase 4: Add CLI and Validation

Deliverables: `dnkzoo generate`; `dnkzoo validate`; and `dnkzoo properties`.

Example:

```
dnkzoo generate \  
  --input topologyzoo/sources/graphml \  
  --topology Arpanet196912 \  
  --scenario diameter-pairwise \  
  --controllers 1 \  
  --seed 15 \  
  --out model.json
```


10.6 Phase 5: Integrate RaceLoom Execution

Deliverables: `dnkzoo run`; `dnkzoo benchmark`; and result-directory format.

Example:

```
dnkzoo run \  
  --model model.json \  
  --properties properties.json \  
  --depth 10 \  
  --strategy bfs
```

10.7 Phase 6: Add Secondary Scenarios

Deliverables: outside-host scenario; link-cost-change scenario; and manual scenario.

10.8 Phase 7: Documentation and Artifact Packaging

Deliverables: README quickstart; architecture document; scenario documentation; schema documentation; benchmark reproduction guide; Dockerfile or devcontainer; and CI workflow.

11 Expected Contributions

This work will contribute a clean benchmark-generation framework for RaceLoom; a documented RaceLoom JSON/property specification; reproducible Topology Zoo benchmark generation; integrated model validation and optional RaceLoom execution; a clearer artifact story for the journal version of the paper; and a migration path from the existing Go generator to a more maintainable Python environment.

12 Risks and Mitigations

Risk	Impact	Mitigation
Subtle mismatch with existing Go generator	Generated benchmarks may differ from paper results	Keep Go generator as reference; add golden tests; compare selected outputs.
Loss of deterministic ordering	Benchmark outputs may not be reproducible	Use ordered data structures; test ordering explicitly.
RaceLoom schema mismatch	Generated models may not parse	Validate every generated model against RaceLoom Pydantic schema.
Over-expanding scope	Project may become too large for journal timeline	Implement diameter-pairwise first; defer append updates and advanced scenarios.
Append-update semantics unsupported	Unsound or misleading scenarios	Avoid append updates initially; document as future work.

RaceLoom integration
issues

End-to-end benchmark
automation may fail

Start with low-depth smoke
tests; preserve direct
RaceLoom CLI compatibility.

13 Advisor Discussion Points

The main decisions to confirm are Should the journal artifact prioritize a robust reconstruction of the paper benchmarks, or a broader benchmark-generation framework?; Is a Python rewrite acceptable, given that the current generator is in Go?; Which scenario should be treated as the primary journal benchmark scenario?; Should append-style flow-rule updates be explicitly out of scope?; Should the generator live as a separate repository or eventually become part of the RaceLoom repository?; and Should benchmark reproduction include automatic RaceLoom execution, or only model/property generation?.